A/B Testing
From Data to Decision

What it is, how it works, and a complete Python pipeline explained end to end

WHAT'S INSIDE

Michal Uhrinek
Data Analytics & AI

A practical learning guide

1. What is A/B testing?

A controlled experiment to decide which of two options actually works better — backed by math, not opinion.

Imagine you have two versions of a web page. Version **A** is what you have now (the "control"). Version **B** has one change — a new button colour, a different headline, a simpler form (the "variant"). You want to know: *does B actually get more people to sign up, or does it just feel like it?*

An A/B test answers this by splitting your visitors randomly into two groups. Group A sees the old version, group B sees the new one. You measure the same thing for both — here, the **conversion rate** (the percentage of users who do what you want, like signing up). Then you use statistics to decide whether any difference is **real** or just **random luck**.

> **The one idea to remember**
>
> Randomly splitting users means the two groups are identical in every way *except* the change you made. So if B performs better, the change is the only thing that could have caused it. That is what makes A/B testing trustworthy.

Why not just compare the numbers directly?

Because small differences happen by chance all the time. If you flip a fair coin 100 times you won't always get exactly 50 heads — you might get 54. A 12% vs 14% conversion difference might be a genuine improvement, or it might be the same kind of luck. A/B testing gives you a number — the **p-value** — that tells you how likely the difference is to be pure chance.

2. The statistics behind it

Four concepts are enough to understand the whole test.

Conversion rate

The fraction of users who converted: `conversions / total users`. If 218 of 1,821 users signed up, the rate is 218 ÷ 1,821 = 11.97%.

Lift — absolute vs relative

Type	Formula	Meaning
Absolute lift	`rate_B − rate_A`	Difference in *percentage points*. 14% − 12% = 2 points.
Relative lift	`(rate_B − rate_A) / rate_A`	Difference *relative* to A. (14−12)/12 = 16.7% better.

Both describe the same gap differently. Executives usually find relative lift more persuasive, but absolute lift is less misleading. Quote both.

The p-value

The probability of seeing a difference *this large or larger* purely by chance, assuming the two versions are actually identical. A small p-value means "this would rarely happen by luck, so the effect is probably real."

> **The 0.05 rule**
>
> By convention, if `p < 0.05` the result is "statistically significant" — we accept B is genuinely different. If `p ≥ 0.05`, we can't rule out chance. The 0.05 threshold (called **alpha**) is a tradition, not a law of nature, but it's the standard across science and industry.

The confidence interval

A range that the *true* lift is likely to fall within (95% of the time). If that range **includes zero**, then "no difference at all" is still plausible — so the result is not conclusive. The number `1.96` in the code is the fixed multiplier that produces a 95% interval on a bell curve.

3. The four-stage pipeline

Every real analysis follows the same shape. Each stage saves a file the next stage reads.

Stage	Script	Input → Output	Job
1. Generate	`generate`	nothing → `raw_data.csv`	Create (or collect) the raw test data
2. Clean	`clean`	`raw_data.csv` → `clean_data.csv`	Remove duplicates, nulls, bad values
3. Analyze	`analyze`	`clean_data.csv` → `results.json`	Compute rates, run the z-test
4. Present	`present`	`results.json` → chart + report	Visualize and write the verdict

Why split it into four files?

Separation of concerns. If your chart looks wrong, you fix stage 4 without re-running the analysis. If new data arrives, you re-run from stage 1. Each stage is small, testable, and re-usable — the same structure professionals use in production data pipelines.

Stage 1 — Generating the data

1 Create realistic raw test data

In a real test this data comes from your website logs. Here we *simulate* it so we have something to practice on — and crucially, we build B to be genuinely better (14% vs 12%), so later we can check whether the analysis correctly detects it.

```
#generating data for A/B testing - conversion rate
import polars as pl
import numpy as np
import os

#settings
n_users = 5000          #number of users per group
rate_a  = 0.12          #A converts 12% of the time (the "true" rate)
rate_b  = 0.14          #B converts 14% of the time -> B is better by design

os.makedirs("data", exist_ok=True)

rng = np.random.default_rng(seed=42)   #seed=42 -> same data every run

#user IDs - narrow range means some IDs repeat (simulates messy real data)
user_ids_a = rng.integers(10000, 14000, size=n_users).tolist()
user_ids_b = rng.integers(10000, 14000, size=n_users).tolist()

#each user converts (1) or not (0); a weighted coin flip per user
converted_a = rng.binomial(1, rate_a, size=n_users).tolist()
variant_a   = ["A"] * n_users
converted_b = rng.binomial(1, rate_b, size=n_users).tolist()
variant_b   = ["B"] * n_users

#combine both groups into one table
df = pl.DataFrame({
    "user_id":   user_ids_a  + user_ids_b,
    "variant":   variant_a   + variant_b,
    "converted": converted_a + converted_b
})

#shuffle so A and B are mixed like a real live test
df = df.sample(fraction=1.0, shuffle=True, seed=42)
df.write_csv("data/raw_data.csv")
```

What the key lines do

- `np.random.default_rng(seed=42)` — a random generator with a fixed **seed**, so you get identical data every run. Essential for reproducibility.
- `rng.binomial(1, rate_a, size=n_users)` — a weighted coin flip per user: 1 = converted, 0 = didn't, with a 12.00% chance of 1.
- `["A"] * n_users` — a label column marking which version each user saw.
- `df.sample(fraction=1.0, shuffle=True)` — shuffles rows so A and B are interleaved, like a live test.

A deliberate quirk: duplicate IDs

The IDs are drawn from only 4,000 possible values (10000–13999) but we create 10,000 rows. By the pigeonhole principle, many IDs *must* repeat. This is intentional — it mimics messy real-world data so Stage 2 has duplicates to clean. As you'll see, this choice has a big consequence for the final result.

Stage 2 — Cleaning & validation

2 Make the data trustworthy before analyzing it

Garbage in, garbage out. Before any statistics, we run five checks. Real analysts spend more time here than anywhere else.

```
#validate and clean the A/B test data
import polars as pl
from scipy import stats

df = pl.read_csv("data/raw_data.csv")

# CHECK 1 - DUPLICATES: keep only the first time we saw each user
duplicates = df.filter(df["user_id"].is_duplicated())
df = df.unique(subset=["user_id"], keep="first")

# CHECK 2 - MISSING DATA: drop any row with a blank value
df = df.drop_nulls()

# CHECK 3 - OUTLIERS: converted must be 0 or 1, nothing else
df = df.filter(pl.col("converted").is_in([0, 1]))

# CHECK 4 - VALID VARIANTS: only A and B allowed
df = df.filter(pl.col("variant").is_in(["A", "B"]))
count_a = df.filter(pl.col("variant") == "A").height
count_b = df.filter(pl.col("variant") == "B").height

# CHECK 5 - 50/50 SPLIT (Sample Ratio Mismatch test)
total = count_a + count_b
expected = total / 2
chi2, srm_p = stats.chisquare([count_a, count_b], [expected, expected])
# srm_p < 0.01 would mean randomization is broken

df.write_csv("data/clean_data.csv")
```

The five checks

#	Check	Why it matters
1	Duplicates	The same user counted twice biases the result. Keep only their first appearance.
2	Missing data	Blank values break calculations. Drop rows with nulls.
3	Outliers / bad values	`converted` must be 0 or 1. Anything else is a data error.
4	Valid variants	Only A and B allowed — no stray "C" or typos.
5	50/50 split (SRM)	If the groups are very unequal, randomization is broken and the test is invalid. The chi-square test flags this.

> **What actually happened here**
>
> Because of the narrow ID range, the duplicate check removed **9,175 of 10,000 rows**, leaving only **3,637**. That's not a bug in the cleaning code — it's a direct result of the data-generation choice in Stage 1. The lesson: *upstream decisions ripple downstream.* A wider ID range (e.g. 10000–99999) would have kept almost all the data.

Stage 3 — Analysis & the z-test

3 Compute the rates and test for significance

```
#compare conversion rates between A and B
import polars as pl
import numpy as np
from scipy import stats

ALPHA = 0.05   #significance threshold

df = pl.read_csv("data/clean_data.csv")

#group A numbers
group_a = df.filter(pl.col("variant") == "A")
n_a, conv_a = group_a.height, group_a["converted"].sum()
rate_a = conv_a / n_a

#group B numbers
group_b = df.filter(pl.col("variant") == "B")
n_b, conv_b = group_b.height, group_b["converted"].sum()
rate_b = conv_b / n_b

#how much better is B?
abs_lift = rate_b - rate_a            #difference in percentage points
rel_lift = abs_lift / rate_a          #difference relative to A

#the two-proportion z-test
p_pool = (conv_a + conv_b) / (n_a + n_b)               #combined rate
se = np.sqrt(p_pool*(1-p_pool) * (1/n_a + 1/n_b))      #standard error
z_stat = (rate_b - rate_a) / se                        #how many SEs from zero
p_value = 2 * (1 - stats.norm.cdf(abs(z_stat)))        #two-sided p-value

#95% confidence interval for the lift
ci_low  = abs_lift - 1.96 * se
ci_high = abs_lift + 1.96 * se
```

The z-test, step by step

The two-proportion z-test answers: "Is the gap between A and B bigger than we'd expect from random noise?"

- `p_pool` — the combined conversion rate if you ignore which group users were in. It's the baseline "if there were no real difference."
- `se` (standard error) — how much the difference would naturally wobble from sample to sample, just by chance.
- `z_stat` — how many standard errors the observed difference sits away from zero. Roughly, |z| above 1.96 suggests a real effect.
- `p_value` — converts the z-score into a probability. The heart of the decision.

Intuition

The standard error shrinks as your sample grows. With huge samples, even a tiny difference becomes significant. With small samples (like our 3,637 rows after cleaning), even a real difference may stay invisible. Sample size is everything.

Stage 4 — Presenting the results

4 Turn numbers into a clear verdict

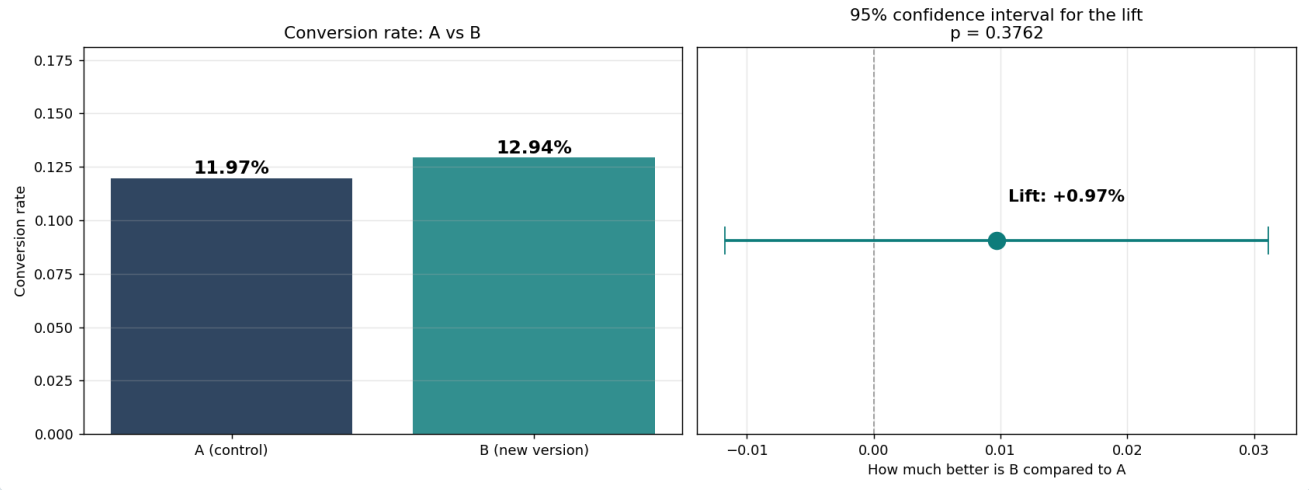
```
#build the chart and the verdict
import matplotlib.pyplot as plt
import json

with open("outputs/results.json") as f:
    r = json.load(f)

#bar chart: A vs B conversion rates  + lift with its confidence interval
#(full plotting code in the project file)

#decide the verdict automatically
if r["significant"] and r["absolute_lift"] > 0:
    verdict = "SHIP IT - B wins and it is statistically confirmed."
elif r["significant"] and r["absolute_lift"] < 0:
    verdict = "DO NOT SHIP - B is significantly worse."
else:
    verdict = "NOT CONCLUSIVE - the difference could be random noise."
```

The presentation step does two jobs: draw a chart a non-technical person can read at a glance, and translate the statistics into a plain-English recommendation (ship / don't ship / inconclusive).



Left: the two conversion rates side by side. Right: the lift with its 95% confidence interval. The dashed line is zero — if the interval crosses it, the result is not conclusive.

Reading the real result

Here is what this pipeline actually produced — and why it's the most useful lesson in the whole guide.

11.97%	12.94%	+0.97%	0.376
A RATE (218/1821)	B RATE (235/1816)	ABSOLUTE LIFT	P-VALUE

Variant	Users	Converted	Rate
A (control)	1,821	218	11.97%
B (new version)	1,816	235	12.94%

Absolute lift: **+0.97%** • Relative lift: **+8.09%** • 95% CI: **[-1.18%, +3.12%]**

Verdict: NOT CONCLUSIVE — cannot call it

The observed lift of +0.97% is **not statistically significant** (p = 0.3762, which is above 0.05). The 95% confidence interval [-1.18%, +3.12%] crosses zero, so we cannot rule out that the true difference is zero — or even negative.

The paradox — and the real lesson

We *built* B to be better (14% vs 12% by design). Yet the test came back **inconclusive**. Why? Because the duplicate removal in Stage 2 shrank the sample from 10,000 to 3,637. With fewer users, the standard error grew, and the test no longer had enough **statistical power** to detect the true 2-point difference. The effect is real — the experiment just couldn't prove it.

This is the single most important thing to understand about A/B testing: **"not significant" does not mean "no difference." It often means "not enough data."**

How to fix it

- **Widen the ID range** in Stage 1 (e.g. 10000–99999) so the duplicate check doesn't destroy the sample. You'd keep ~10,000 rows and the true effect would likely become significant.
- **Run a power analysis before testing.** To reliably detect a 2-point lift on a 12% baseline at 80% power, you need roughly 4,000–4,500 users *per variant*. Plan the sample size up front.
- **Let the test run longer** in real life — more traffic means more power.

Five rules to take away

Rule	Why
Decide sample size before you start	Underpowered tests waste effort and mislead.
Don't "peek" and stop early	Checking repeatedly until you see significance inflates false positives.
Always report the confidence interval	A p-value alone hides how uncertain the estimate is.
Significant ≠ important	A tiny lift can be significant with huge samples; check the lift size too.
Clean before you analyze	As we saw, cleaning choices can change the entire conclusion.

Where this fits

A/B testing is hypothesis testing applied to a product decision. The same z-test logic powers everything from website optimization to the kind of operational change you ran in Prague — comparing accuracy before and after the four-eyes principle is the same statistical question: *is this difference real, or luck?*